Funções em Python

def, parâmetros, argumentos e retorno

Aula 15 • 26/10/2026 • Linguagem de Programação Python (80h) • CETAM
Profa. Priscila Gonçalves

Visão geral da aula

Data

Segunda-feira, 26/10/2026
Aula 15 • 4 tempos
Laboratório de informática

Habilidade técnica

Aplicar o recurso de modularização e métodos para o uso de funções e parâmetros.

Projeto

Refatorar o sistema em funções

Habilidade técnica

- Aplicar o recurso de modularização e métodos para o uso de funções e parâmetros
- Criar e chamar funções com def
- Passar parâmetros e usar retorno
- Reaproveitar código em todo o sistema

Objetos de conhecimento

- Funções em Python: criação e chamada de funções
- Passagem de parâmetros e retorno
- Definindo funções com def()
- Parâmetros, argumentos e retorno

Roteiro dos 4 tempos

1º tempo

Aula expositiva: criação e chamada de funções com def()

2º tempo

Devolutiva da AV3, parâmetros e retorno

3º tempo

Por que modularizar, anatomia de def e atividade avaliativa

4º tempo

Desafio: refatorar o sistema em funções

1

1º tempo • Funções e def

Criação e chamada de funções

Retomada: onde estamos?

- Aulas 11 a 13: listas, tuplas, dicionários e strings
- Aula 14: AV3 sobre estruturas de dados
- Fase 5: modularização do sistema
- Até agora o código fica todo em sequência
- Hoje: dividir o programa em funções

Funções que já usamos

- print(): mostra na tela
- input(): lê o que a pessoa digita
- len(): conta os itens
- range(): gera sequências
- int() e float(): convertem tipos

O que é uma função?

- Função: bloco de código com nome
- Executa uma tarefa específica
- Definida uma vez, usada várias vezes
- Definimos com def e chamamos pelo nome
- Torna o programa mais organizado

Primeira função

```
def saudacao():
    print("Olá, Amazonas!")

saudacao()
saudacao()
```

- def define a função
- Chamar executa o bloco
- Pode chamar várias vezes

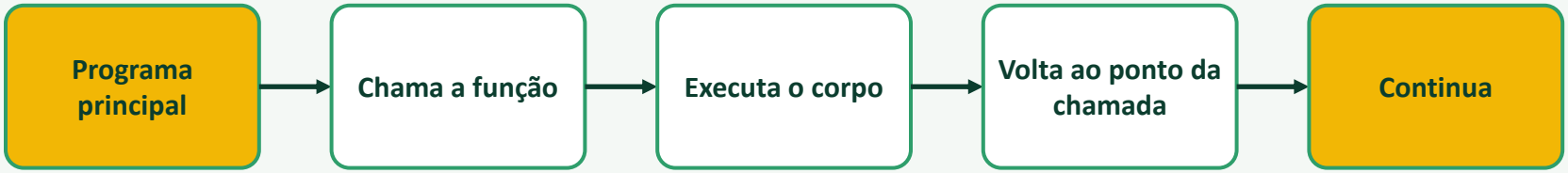
Definir não é executar

```
def exibir_titulo():
    print("=== Agência ===")

print("Início")
exibir_titulo()
print("Fim")
```

- Saída: Início, título e Fim
- Definir não executa
- Chamar executa

O caminho de uma chamada



Cada chamada executa o corpo da função e retorna ao ponto de onde saiu.

Defina antes de chamar

```
saudacao()   # NameError

def saudacao():
    print("Olá!")
```

- O Python lê de cima para baixo
- Chamar antes de definir: NameError
- Funções ficam no topo

O menu como função

```
def exibir_menu():
    print("1 - Calcular viagem")
    print("2 - Ver destinos")
    print("0 - Sair")

exibir_menu()
```

- Trecho do menu da Aula 10
- Agora reutilizável
- Chame sempre que precisar

Nomes e boas práticas

- Nome de verbo: calcular_total, exibir_menu
- Letras minúsculas e sublinhado (snake_case)
- Sem acentos nem espaços
- Uma função, uma tarefa
- Nome que explique o que a função faz

Erros comuns

- Esquecer os dois-pontos depois de def
- Corpo da função sem indentação
- Chamar sem parênteses: saudacao não executa
- Chamar a função antes de defini-la
- Dar o mesmo nome a duas funções

Função com decisão

```
def classificar(total):
    if total <= 1000:
        print("Econômico")
    else:
        print("Conforto ou Premium")

classificar(800)
```

- A função pode ter if, for e while
- Saída: Econômico
- Mas ainda usa um valor fixo

2

2º tempo • Parâmetros e retorno

Devolutiva da AV3 e modularização

Devolutiva da AV3

- Devolutiva qualitativa, individual e respeitosa
- Pontos fortes de cada prova
- Pontos de melhoria com exemplos
- Erros frequentes tratados em conjunto, sem expor ninguém
- Objetivo: aprender com o erro e seguir em frente

Pontos de atenção da Fase 4

- Índices e chaves digitados com atenção
- Tipos de input() convertidos
- Resultado de método de string guardado em variável
- Dicionários e listas combinados com cuidado
- Leitura atenta do enunciado

Parâmetros e argumentos

- Parâmetro: variável na definição da função
- Argumento: valor passado na chamada
- def saudar(nome): nome é o parâmetro
- saudar("Ana"): "Ana" é o argumento
- Parâmetros tornam a função reaproveitável

Função com um parâmetro

```
def saudar(nome):
    print("Olá,", nome)

saudar("Ana")
saudar("Caio")
```

- nome é o parâmetro
- "Ana" e "Caio" são argumentos
- Mesma função, resultados diferentes

Vários parâmetros

```
def mostrar_total(dias, gasto):
    print(dias * gasto)

mostrar_total(4, 150)
mostrar_total(7, 200)
```

- Saída: 600 e 1400
- Argumentos na mesma ordem
- Separados por vírgula

Retorno com return

```
def calcular_total(dias, gasto):
    return dias * gasto

total = calcular_total(4, 150)
print(total)   # 600
```

- return devolve um valor
- Dá para guardar em uma variável
- A função encerra no return

print x return

- print mostra o valor na tela
- return devolve o valor para quem chamou
- Sem return, a função devolve None
- Com return, podemos guardar e calcular
- Regra: funções de cálculo usam return

Parâmetro com valor padrão

```
def desconto(preco, taxa=0.1):
    return preco * (1 - taxa)

print(desconto(100))        # 90.0
print(desconto(100, 0.2))    # 80.0
```

- taxa tem valor padrão 0.1
- Sem o argumento, usa o padrão
- Com o argumento, usa o informado

3

3º tempo • Modularizar

Por que, anatomia de def e atividade avaliativa

Por que modularizar?

- Evita repetir código
- Facilita corrigir: muda em um só lugar
- Deixa o programa mais legível
- Permite testar cada parte separadamente
- Facilita o trabalho em equipe

Sem função: código repetido

```
print("=" * 20)
print("Pacote Econômico")
print("=" * 20)

print("=" * 20)
print("Pacote Premium")
print("=" * 20)
```

- Mesmo bloco repetido
- Mudar um exige mudar todos
- A função resolve isso

Com função: sem repetição

```
def titulo(texto):
    print("=" * 20)
    print(texto)
    print("=" * 20)

titulo("Pacote Econômico")
titulo("Pacote Premium")
```

- Bloco escrito uma vez
- O texto vira parâmetro
- Mudou? Muda em um lugar só

Anatomia de uma função

def

Palavra que inicia a definição

Nome e parâmetros

Nome da função e dados de entrada

corpo

Linhas recuadas que fazem o trabalho

return

Devolve o resultado (opcional)

Chamada e reaproveitamento

- Defina uma vez, chame quantas vezes quiser
- Cada chamada pode usar argumentos diferentes
- Funções podem chamar outras funções
- Reaproveite funções em outros programas
- Resultado: menos código e menos erros

Atividade avaliativa: refatorar com funções

- Escolha trechos repetidos dos exercícios anteriores
- Exemplos: validação de dias, média, troco e desconto
- Transforme cada trecho em uma função com parâmetros
- Entrega: arquivo .py com chamadas de teste
- Critérios: nomes claros, parâmetros e return corretos

Exemplo: validação vira função

```
def ler_dias():
    while True:
        dias = int(input("Dias: "))
        if 1 <= dias <= 30:
            return dias
        print("Valor inválido!")
```

- Validação da Aula 10 em função
- return sai do laço e da função
- Use: dias = ler_dias()

4

4º tempo • Desafio do projeto

Refatorar o sistema em funções

Desafio: refatorar o sistema

- Refatorar o sistema da agência em funções
- calcular_total(dias, gasto): devolve o total da viagem
- exibir_menu(): mostra as opções do menu
- cadastrar_cliente(): registra um cliente na lista
- O programa principal só chama as funções

Exemplo: as três funções

```
def calcular_total(dias, gasto):
    return dias * gasto

def exibir_menu():
    print("1 - Calcular viagem")
    print("2 - Cadastrar cliente")
    print("0 - Sair")

def cadastrar_cliente(clientes):
    nome = input("Nome: ").strip().title()
    clientes.append({"nome": nome})
```

- Cada função faz uma tarefa
- cadastrar_cliente recebe a lista
- Ajuste as opções ao seu menu

Exemplo: programa principal

```
clientes = []
while True:
    exibir_menu()
    op = input("Escolha: ")
    if op == "0":
        break
    elif op == "1":
        d = int(input("Dias: "))
        g = float(input("Gasto/dia: "))
        print(calcular_total(d, g))
    elif op == "2":
        cadastrar_cliente(clientes)
```

- Programa principal enxuto
- Compare com o menu da Aula 10
- As funções ficam acima

Dicas para o desafio

- Mova um bloco por vez e teste a cada passo
- Defina as funções antes do programa principal
- Dê nomes que expliquem a tarefa
- Prefira return a print dentro de calcular_total
- Compare o resultado com a versão original

Fechamento

- Hoje: funções, parâmetros e retorno
- Atividade: trechos repetidos viram funções
- Desafio: calcular_total(), exibir_menu() e cadastrar_cliente()
- Dúvidas?
- Obrigada!